ШПАРГАЛКА ДО ЗАХИСТУ ЗА 2 ГОДИНИ

Лабораторна робота №6: React.js (Home Page)

Предмет: Web-розробка **Варіант:** 8 (Helicopter / Вертольоти) **Стек:** React 18, JSX, Functional Components, CSS Modules

1. Головна суть: Чому React і що змінилося після 5-ї лаби?

Головна думка, яку треба сказати викладачу:

*«У 5-й лабораторній ми писали окремо HTML-розмітку у файлі `data/index.html`, а в JavaScript вручну шукали елементи через `document.getElementById` і склеювали рядки через `innerHTML` (**імперативний підхід**). У React ми перейшли на **декларативний підхід** та синтаксис **JSX**: тепер розмітка та логіка компонента описані разом в одному файлі JavaScript, а оновленням DOM займається сам React через механізм Virtual DOM».*

Критерій	Як було в Лабі 3–5 (Vanilla JS)	Як стало в Лабі 6 (React.js)
Файли верстки	Окремий великий `data/index.html` зі структурою тегів.	`public/index.html` містить лише `<div id="root"></div>`. Увесь HTML генерується в React.
Формування карток	Склеювання шаблонних рядків `items.map(h ⇒ ` ... `).join('')` та `innerHTML = ...`.	Функціональний компонент `<HelicopterCard helicopter={heli} />`, який повертає **JSX**.
Поділ коду	Глобальні скрипти (`dom.js`, `main.js`, `api.js`), які взаємодіють з глобальним DOM.	Компонентна архітектура: кожен візуальний блок (Header, Sidebar, Card) — це окремий незалежний компонент у власному файлі.
Стилізація	Окремі CSS файли, що підключалися через тег `<link>` в HTML.	Модульний імпорт CSS безпосередньо у відповідний JS-файл компонента (`import './Header.css'`).

2. Архітектура файлів нашого проєкту

За вимогою завдання інтерфейс розділено щонайменше на 5 окремих функціональних компонентів (кожен у своєму файлі):

1. Header (Шапка)

`src/components/Header/Header.jsx`

Бренд **MyHeli**, навігація (My Helicopters, Create), поле пошуку та кнопки Search/Clear.

2. Sidebar (Бокова панель)

`src/components/Sidebar/Sidebar.jsx`

Керування вертольотами: перемикач сортування (Sort toggle) та кнопка підрахунку пасажирів (Count).

3. CardsGrid (Сітка карток)

`src/components/CardsGrid/CardsGrid.jsx`

Контейнер секції контенту. Проходиться методом `.map()` по масиву даних вертольотів.

4. HelicopterCard (Картка вертольота)

`src/components/HelicopterCard/HelicopterCard.jsx`

Індивідуальна картка (235x180 прев'ю, назва, опис, характеристики, кнопки Edit/Remove).

5. Footer (Підвал сайту)

`src/components/Footer/Footer.jsx`

Інформація про систему MyHeli, дублюючі лінки та копірайт.

6. App (Головний контейнер)

`src/App.jsx`

Кореневий компонент, який імпортує дані вертольотів та збирає всі компоненти на одній сторінці.

3. Покроковий розбір коду (Що відбувається в кожному файлі)

3.1. Точка монтування: `public/index.html` та `src/index.js`

У `public/index.html` немає жодного нашого тегу, крім:

```
<div id="root"></div>
```

У `src/index.js` React підключається до цього контейнера:

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';

// Створюємо корінь React 18 через новий метод createRoot
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Що сказати: `ReactDOM.createRoot` знаходить звичайний DOM-вузол з `id="root"` і бере над ним повний контроль. Метод `render(<App />)` перетворює дерево компонентів React на реальні DOM-вузли всередині цього блоку.

3.2. Головний компонент `src/App.jsx` та джерело даних

Дані про вертольоти зберігаються у `src/data/helicopters.js` (масив з 5 об'єктів з полями `id, name, passengers, maxSpeed, description`). Компонент `App` збирає всю сторінку:

```
import React from 'react';
import Header from './components/Header/Header';
import Sidebar from './components/Sidebar/Sidebar';
import CardsGrid from './components/CardsGrid/CardsGrid';
import Footer from './components/Footer/Footer';
import { INITIAL_HELICOPTERS } from './data/helicopters';
import './App.css';

function App() {
  return (
    <div className="app">
      <Header />
      <main className="main-container">
        <Sidebar totalPassengers="—" isSorted={false} />
        {/* Передаємо масив вертольотів дочірньому компоненту через props */}
        <CardsGrid helicopters={INITIAL_HELICOPTERS} />
      </main>
      <Footer />
    </div>
  );
}

export default App;
```

3.3. Рендеринг списку: `src/components/CardsGrid/CardsGrid.jsx`

```
import React from 'react';
import HelicopterCard from '../HelicopterCard/HelicopterCard';
import './CardsGrid.css';

const CardsGrid = ({ helicopters = [] }) => {
  return (
    <section className="content">
      <div className="cards-grid" id="cards-grid">
        {helicopters.length === 0 ? (
          <div className="cards-empty">Вертольотів не знайдено.</div>
        ) : (
          /* Метод .map() перетворює кожен JS-об'єкт вертольота на JSX-компонент */
          helicopters.map((heli) => (
            <HelicopterCard key={heli.id} helicopter={heli} />
          ))
        )}
      </div>
    </section>
  );
};

export default CardsGrid;
```

3.4. Відображення одного вертольота: `src/components/HelicopterCard/HelicopterCard.jsx`

```
import React from 'react';
import './HelicopterCard.css';

const HelicopterCard = ({ helicopter }) => {
  // Деструктуризація об'єкта пропса для зручного звернення
  const { id, name, description, maxSpeed, passengers } = helicopter;

  return (
    <article className="card" data-id={id}>
      <div className="card__image">
        <span className="card__image-text">235x180</span>
      </div>
      <div className="card__body">
        <h3 className="card__title">{name}</h3>
        <p className="card__desc">{description}</p>
        <p className="card__specs">
          <strong>Макс. швидкість:</strong> {maxSpeed} км/год<br />
          <strong>Пасажири:</strong> {passengers} місць
        </p>
        <div className="card__updated">Last updated 3 mins ago</div>
        <div className="card__actions">
          <button type="button" className="btn-card-edit">Edit</button>
          <button type="button" className="btn-card-remove">Remove</button>
        </div>
      </div>
    </article>
  );
};

export default HelicopterCard;
```

4. ТОП-11 канонічних питань захисту (Questionnaire)

QUESTION 1 **What is React? (Що таке React?)**

Answer: React is an open-source JavaScript front-end library developed by Meta (Facebook) for building user interfaces, especially Single Page Applications (SPAs). It is based on a **component-based architecture**, uses a **Virtual DOM** for high-performance rendering, and enforces a **declarative approach** where the UI is expressed as a pure function of state and props (`UI = f(state, props)`).

QUESTION 2 **What is Single Page Application (SPA)? (Що таке SPA?)**

Answer: An **SPA** is a web application that loads only a single initial HTML file (`public/index.html` with `<div id="root"></div>`). As the user navigates, JavaScript dynamically rewrites the existing page with new content rather than downloading entire new HTML pages from the server. This provides a fluid, desktop-like user experience with zero full-page browser reloads.

QUESTION 3 **What are the major features of React? (Основні особливості React)**

Answer:

- **JSX:** Syntax extension allowing HTML-like markup inside JavaScript.
- **Virtual DOM:** Lightweight in-memory DOM representation with efficient diffing algorithm.
- **Component-Based Architecture:** Autonomous, modular, and reusable UI building blocks.
- **One-Way Data Flow:** Unidirectional data flow from parent to child via `props`, ensuring predictable state.
- **Declarative UI:** You define *what* the UI should look like; React handles *how* to update the DOM.
- **React Hooks:** Functional primitives like `useState` and `useEffect` for managing state and lifecycles.

QUESTION 4 **What is JSX? (Що таке JSX?)**

Answer: JSX stands for *JavaScript XML*. It is a syntax extension for JavaScript that allows you to write HTML-like structure directly inside JS files. Browsers cannot read JSX directly; a compiler like **Babel** transpile it into standard `React.createElement()` (or `_jsx()`) function calls that produce JavaScript objects. In JSX, JS expressions are embedded inside curly braces `{ }`, and HTML attributes use DOM properties like `className` and `htmlFor`.

QUESTION 5 **What is the difference between Element and Component? (Різниця між Element і Component)**

Answer:

- **React Element:** A plain, immutable JavaScript object that describes what you want to see on screen (e.g. `const el = <h1>Hello</h1>;`). It is an ephemeral snapshot of a DOM node.
- **React Component:** A reusable function (or class) that accepts inputs (`props`) and returns a React element tree (e.g. `function HelicopterCard(props) { return <article>...</article>; }`). Components are factories/blueprints for producing elements.

QUESTION 6 **How to create components in React? (Як створювати компоненти в React?)**

Answer: There are two ways:

1. **Functional Components (Modern standard):** Regular JS functions that take `props` as an argument and return JSX:

```
function HelicopterCard({ helicopter }) {
  return <h3>{helicopter.name}</h3>;
}
```

2. **Class Components (Legacy):** ES6 classes extending `React.Component` with a `render()` method:

```
class HelicopterCard extends React.Component {
  render() { return <h3>{this.props.helicopter.name}</h3>; }
}
```

QUESTION 7 **What is state in React? (Що таке state?)**

Answer: State is an internal, private data store managed entirely within a component. Unlike static variables, when a component's state changes (via `useState` updater), React automatically detects the change and triggers a **re-render** of the component to update the UI with new information.

QUESTION 8 **What are props in React? (Що таке props?)**

Answer: Props (short for "properties") are read-only configuration inputs passed from a parent component down to a child component (e.g., `<CardsGrid helicopters={INITIAL_HELICOPTERS} />`). Props are **immutable**: a child component must never modify its received props.

QUESTION 9 **What is the difference between state and props? (Різниця між state та props)**

Answer:

Критерій	Props (Властивості)	State (Стан)
Походження	Передаються ззовні (від батьківського компонента).	Створюється та керується всередині самого компонента.
Змінюваність	**Read-only (незмінні)** для компонента-отримувача.	**Змінюваний** через спеціальну функцію (наприклад, `setState`).
Призначення	Конфігурація дочірнього компонента, передача даних зверху вниз.	Зберігання динамічних внутрішніх даних, які змінюються з часом.

QUESTION 10 **Why should we not update the state directly? (Чому не можна змінювати state напряму?)**

Answer: If you mutate state directly (e.g., `state.count = 5`), React has **no way of detecting the mutation**. As a result:

- No re-render will be triggered, leaving the UI out of sync with the data.
- It breaks React's batching and reconciliation optimizations.
- It causes unpredictable side effects and race conditions.

State must always be modified via its setter function (e.g. `setCount(5)`), which schedules a proper re-render.

QUESTION 11 **What happens when you update the state? (Що відбувається при оновленні state?)**

Answer: When state is updated via a setter function:

1. **Schedule update:** React enqueues the state update and schedules a re-render.
2. **Re-render:** React invokes the component function again with the updated state values.
3. **New Virtual DOM:** A new Virtual DOM tree representing the new UI is generated.
4. **Diffing:** React compares the new Virtual DOM tree with the previous snapshot.
5. **Reconciliation (Commit):** React calculates the minimal set of real DOM operations and patches only the changed real DOM nodes in the browser window, leaving the rest untouched.

5. Експрес-пам'ятка за 2 хвилини до захисту

Якщо викладач запитає «Розкажи коротко свій код»:

Говори за цією схемою (30 секунд):

1. «У цій лабораторній роботі ми перенесли наш сайт каталогу вертольотів з ванільного JS на **React 18**».
2. «Ми повністю зберегли верстку та дизайн із 5-ї лаби (шапка `MyHeli`, бокове меню сортування та сітка карток), але тепер весь UI розбитий на **5 окремих функціональних компонентів**: `Header`, `Sidebar`, `CardsGrid`, `HelicopterCard` та `Footer`».
3. «У `index.html` розмітка чиста (тільки `div id="root"`). Точка входу `index.js` монтує кореневий компонент `App` за допомогою `createRoot`».
4. «Дані вертольотів зберігаються у масиві `INITIAL_HELICOPTERS` і передаються через `props` у `CardsGrid`, де за допомогою методу `.map()` рендеряться окремі картки `HelicopterCard` з унікальним `key={heli.id}`».
5. «Усі компоненти функціональні, без класового `this`. Згідно з вимогами лаби 6, реалізовано представлення (view-only), а стан та взаємодію буде додано в наступних роботах».
